Fast Fourier Transform (FFT)

Bikash Samanta
Indian Statistical Institute, Kolkata

1 The Coefficient Representation

To understand how FFT optimizes polynomial multiplication, we must first examine how polynomials are represented and the limitations of the standard approach.

The most intuitive way to represent a polynomial is by its coefficients. We can map the terms of a polynomial to an array (or list) where the index k corresponds to the degree of the term.

Example:
Given two polynomials:

$$A(x) = x^2 + 3x + 2$$
$$B(x) = 2x^2 + 1$$

We can represent them as arrays where the kth index holds the coefficient for the x^k term:

- $A = [2, 3, 1]$
- $B = [1, 0, 2]$

If we want to compute $C(x) = A(x) \cdot B(x)$ algebraically, we distribute the terms:

$$C(x) = 2x^4 + 6x^3 + 5x^2 + 3x + 2$$

In array format, the resulting polynomial C is represented as:

- $C = [2, 3, 5, 6, 2]$

In this representation, any element $C[k]$ is exactly the coefficient of the kth degree term of the polynomial $C(x)$.

Generalization & Time Complexity

For two polynomials of degree d:

$$A(x) = a_0 + a_1x + a_2x^2 + \cdots + a_dx^d$$
$$B(x) = b_0 + b_1x + b_2x^2 + \cdots + b_dx^d$$

The resulting polynomial $C(x)$ will have a degree of $2d$:

$$C(x) = c_0 + c_1x + c_2x^2 + \cdots + c_{2d}x^{2d}$$

When utilizing the standard distributive property to multiply these polynomials in code, every term in $A(x)$ must be multiplied by every term in $B(x)$.

- **Runtime using distributive property:** $\mathcal{O}(d^2)$

---

2 The Value Representation

To bypass the $\mathcal{O}(d^2)$ bottleneck of coefficient multiplication, we introduce an alternative way to define a polynomial.

Core Theorem:

$(d+1)$ points uniquely define a degree d polynomial.

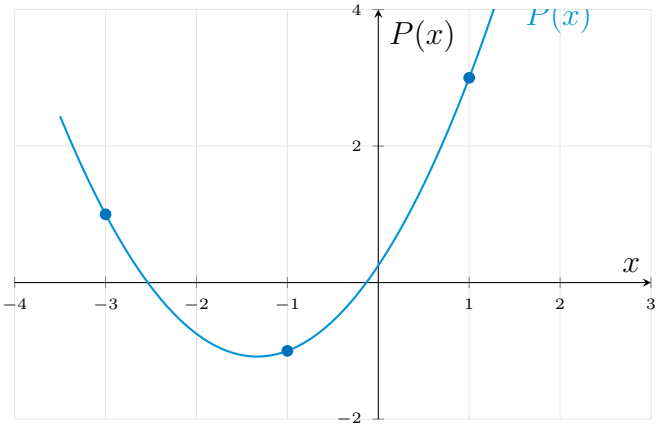
Instead of storing the coefficients, a polynomial can be perfectly represented by a set of coordinates evaluated at distinct values of x.

Example:
Given a set of 3 points (which can uniquely define a degree $d = 2$ polynomial):

$$\{(-3, 1), (-1, -1), (1, 3)\}$$

These points uniquely map to the following polynomial:

$$P(x) = \frac{3}{4}x^2 + 2x + \frac{1}{4}$$



Generalization & Matrix Formulation

For a polynomial $P(x) = p_0 + p_1x + p_2x^2 + \cdots + p_dx^d$, we can define its value representation as a set of $d+1$ points:

$$\{(x_0, P(x_0)), (x_1, P(x_1)), \ldots, (x_d, P(x_d))\}$$

This relationship between the evaluated points and the unknown coefficients can be expressed as a system of linear equations using matrix multiplication:

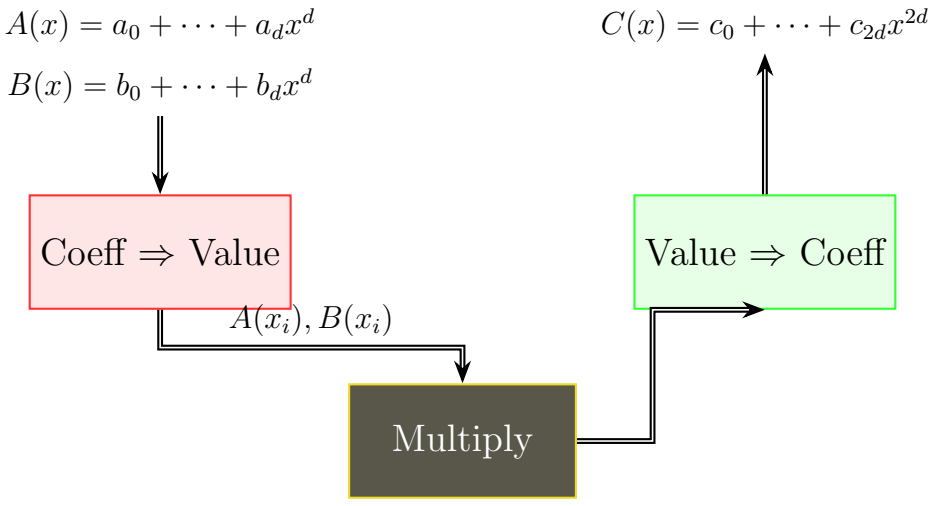
$$\begin{bmatrix} P(x_0) \\ P(x_1) \\ \vdots \\ P(x_d) \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & x_0 & x_0^2 & \ldots & x_0^d \\ 1 & x_1 & x_1^2 & \ldots & x_1^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_d & x_d^2 & \ldots & x_d^d \end{bmatrix}}_{M} \begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_d \end{bmatrix}$$

The matrix M (known in linear algebra as a Vandermonde matrix) transforms the coefficient representation into the value representation. Crucially, M **is invertible for unique** $x_0, x_1, \ldots, x_d$. This invertibility guarantees that we can always translate back and forth between the Value Representation and the Coefficient Representation without losing information.

3 The Multiplication Workflow

Why do we care about the Value Representation? Because multiplying two polynomials in this form is incredibly fast. If we have the value representations of $A(x)$ and $B(x)$ at the same $2d+1$ points, computing $C(x) = A(x)B(x)$ only requires multiplying their corresponding y-values: $\mathcal{O}(d)$ time.

The overall optimized algorithm for polynomial multiplication follows this pipeline:



4 Evaluation Strategy & Computational Overlap

To make the workflow above efficient, we must evaluate $P(x)$ at $n \geq d+1$ points quickly. The standard evaluation approach is too slow:

$$\left.\begin{array}{c}(1, p_0 + p_1 \cdot 1 + p_2 \cdot 1^2 + \cdots + p_d \cdot 1^d) \\ (2, p_0 + p_1 \cdot 2 + p_2 \cdot 2^2 + \cdots + p_d \cdot 2^d) \\ \vdots \\ (n, p_0 + p_1 \cdot n + p_2 \cdot n^2 + \cdots + p_d \cdot n^d)\end{array}\right\} \mathcal{O}(nd) \Leftrightarrow \mathcal{O}(d^2)$$

Exploiting Symmetry (Even and Odd Functions)

If we smartly pick our x points as positive and negative pairs (e.g., $\pm x_1, \pm x_2, \ldots, \pm x_{n/2}$), we can exploit the symmetry of polynomials to drastically reduce computations.

For purely even or odd terms, calculating $P(x)$ inherently gives us $P(-x)$:

- **Even terms (e.g., x^2):** $P(-x) = P(x)$
- **Odd terms (e.g., x^3):** $P(-x) = -P(x)$

The Even/Odd Decomposition Trick

Consider a general polynomial:

$$P(x) = 3x^5 + 2x^4 + x^3 + 7x^2 + 5x + 1$$

We can group the even-powered terms and the odd-powered terms, factoring out an x from the odd group:

$$P(x) = \underbrace{(2x^4 + 7x^2 + 1)}_{\text{Even powers}} + \underbrace{x(3x^4 + x^2 + 5)}_{\text{Odd powers}}$$

Let us define two new polynomials, P_e and P_o, constructed from the coefficients of the even and odd terms respectively. Both are evaluated at x^2:

$$P_e(x^2) = 2(x^2)^2 + 7(x^2) + 1$$
$$P_o(x^2) = 3(x^2)^2 + 1(x^2) + 5$$

Notice that both P_e and P_o are now only degree 2 polynomials (roughly half the size of the original degree 5 polynomial). We can express our original polynomial as:

$$P(x) = P_e(x^2) + xP_o(x^2)$$

We can generalize this even/odd decomposition for a polynomial of degree $n-1$:

$$P(x) = p_0 + p_1 x + p_2 x^2 + \cdots + p_{n-1} x^{n-1}$$

If we evaluate at n points structured as $\pm x_1, \pm x_2, \ldots, \pm x_{n/2}$, we split the polynomial as before:

$$P(x) = P_e(x^2) + xP_o(x^2)$$

This gives us the highly efficient paired evaluation:

$$P(x_i) = P_e(x_i^2) + x_i P_o(x_i^2)$$
$$P(-x_i) = P_e(x_i^2) - x_i P_o(x_i^2)$$

Notice that $P_e(x^2)$ and $P_o(x^2)$ now have a degree of $n/2 - 1$.

To compute the final values, we simply need to evaluate $P_e(x^2)$ and $P_o(x^2)$ each at the $n/2$ points: $x_1^2, x_2^2, \ldots, x_{n/2}^2$. This means we have reduced our original problem of evaluating a degree $n-1$ polynomial into the **same process on a simpler problem** (two polynomials of half the degree).

---

5 The Real Number: Recursion Breaks!

While the divide-and-conquer approach looks perfect, there is a massive mathematical roadblock if we restrict ourselves to real numbers.

For the recursion to continue seamlessly down the tree, the input points at *every* level must be paired as positive and negative values ($\pm x$). Let's trace the points through one step of the recursion:

- **Level 1 Input Points:** $[\pm x_1, \pm x_2, \ldots, \pm x_{n/2}]$. These **are** $\pm$ paired.
- **Level 2 Input Points:** $[x_1^2, x_2^2, \ldots, x_{n/2}^2]$.

Here lies the problem: any real number squared is strictly positive. Therefore, the points $[x_1^2, x_2^2, \ldots, x_{n/2}^2]$ **are not $\pm$ paired**.

Because the next level of evaluation lacks these crucial $\pm$ pairings, the symmetric overlap trick falls apart. **The recursion breaks!**

The Solution

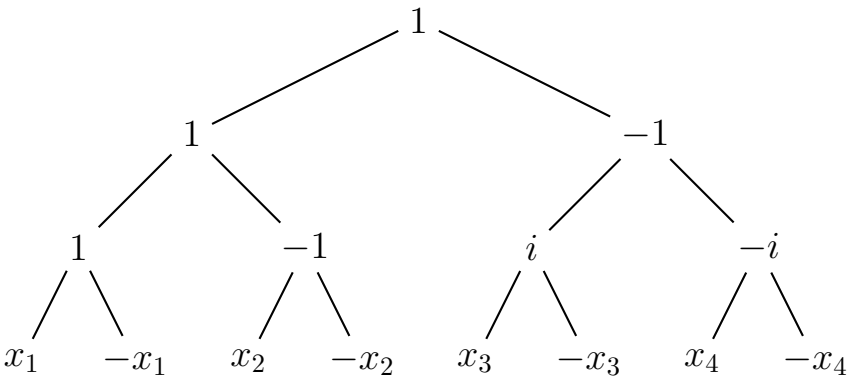
To fix this and keep the recursion going all the way down, we must ask: *Is it possible to make the squared array $[x_1^2, x_2^2, \ldots, x_{n/2}^2]$ $\pm$ paired?*

Yes, but only if some of the original evaluation points $[\pm x_1, \pm x_2, \ldots, \pm x_{n/2}]$ are **complex numbers**. This is the exact reason why the Fast Fourier Transform operates over the complex plane using the roots of unity.

6 The Complex Domain & Roots of Unity

To ensure our recursion does not break, we need a set of $n \geq (d+1)$ points (where we conveniently choose $n = 2^k$ for some integer k) that possess a magical property: when we square them, the resulting set of points is still $\pm$ paired.

If we trace this requirement backward from our base case, we see why we are forced into the complex plane:



Notice that squaring 1 and -1 yields 1. Squaring i and $-i$ yields -1. These points are the nth **roots of unity**! They are the solutions to the equation:

$$z^n = 1$$

Using Euler's formula, $e^{i\theta} = \cos(\theta) + i\sin(\theta)$, we can define the fundamental n^{th} root of unity as:

$$\omega = e^{\frac{2\pi i}{n}}$$

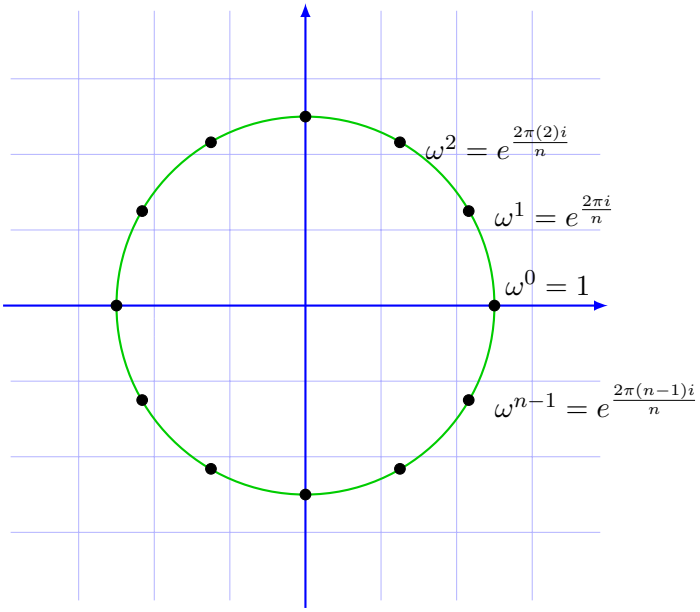
The geometry of the roots of unity perfectly satisfies our divide-and-conquer requirement. Because the roots are equally spaced around the unit circle, points that are 180° apart are exact negations of each other.

Mathematically, this translates to:

$$\omega^{j+n/2} = -\omega^j$$

This proves that the set of points $(\omega^j, \omega^{j+n/2})$ are strictly $\pm$ **paired**.

When we move down a recursive step and square these points, the n roots of unity map perfectly onto the $n/2$ roots of unity. The pairing is preserved at every single level until we hit the base case!



The n points we must use to evaluate our polynomial are exactly the n powers of ω:

$$\text{Evaluate } P(x) \text{ at } [1, \omega^1, \omega^2, \ldots, \omega^{n-1}]$$

---

7 The FFT Algorithm

Here is the recursive implementation of the Fast Fourier Transform translating the coefficient representation of a polynomial into its value representation in $\mathcal{O}(n \log n)$ time.

Algorithm 1 Fast Fourier Transform (FFT)

```
1:  function FFT(P)
2:                                  ▷ P = [p_0, p_1, ..., p_{n-1}] coefficient representation
3:      n ← len(P)
4:                                  ▷ n is assumed to be a power of 2
5:      if n = 1 then
6:          return P
7:      end if
8:      ω ← e^{2πi/n}
9:      P_e ← P[0 :: 2]             ▷ Extract even-indexed coefficients
10:     P_o ← P[1 :: 2]             ▷ Extract odd-indexed coefficients
11:     y_e ← FFT(P_e)
12:     y_o ← FFT(P_o)
13:     y ← [0] × n                 ▷ Initialize output array of size n
14:     for j ← 0 to n/2 − 1 do
15:         y[j] ← y_e[j] + ω^j y_o[j]
16:         y[j + n/2] ← y_e[j] − ω^j y_o[j]
17:     end for
18:     return y
19: end function
```

8 Time Complexity Analysis

The time complexity of the Fast Fourier Transform can be determined by analyzing its recursive structure. We can express the total runtime, $T(n)$, as a recurrence relation based on the steps in the algorithm.

- **Base Case (Lines 5-7):** When $n = 1$, the algorithm simply returns the array. This takes constant time, $\mathcal{O}(1)$.
- **Divide (Lines 9-10):** Splitting the array of coefficients P into even-indexed (P_e) and odd-indexed (P_o) subarrays of size $n/2$ requires a linear scan, taking $\mathcal{O}(n)$ time.

- **Conquer (Lines 11-12):** The algorithm makes two recursive calls to itself, each with an input size of $n/2$. This yields $2T(n/2)$.
- **Combine (Lines 14-17):** The `for` loop iterates $n/2$ times. Inside the loop, we perform basic arithmetic operations (multiplication, addition, and subtraction) which take constant $\mathcal{O}(1)$ time per iteration. Thus, the total time for the combination step is bounded by $\mathcal{O}(n)$.

The Recurrence Relation

Summing the work done at each step, we obtain the following recurrence relation for the total runtime $T(n)$:

$$T(n) = 2T\left(\frac{n}{2}\right) + \mathcal{O}(n)$$
$$T(1) = \mathcal{O}(1)$$

Solving the Recurrence

We can solve this recurrence relation using the **Master Theorem** for divide-and-conquer recurrences of the form $T(n) = aT(n/b) + f(n)$.

Here, $a = 2$, $b = 2$, and $f(n) = \mathcal{O}(n)$. We compute the critical exponent:

$$c_{\text{crit}} = \log_b a = \log_2 2 = 1$$

Since $f(n) = \mathcal{O}(n^1)$, we fall exactly into Case 2 of the Master Theorem, where the work done outside the recursive calls is proportional to the number of leaves in the recursion tree. Therefore, we multiply the work per level by the height of the tree ($\log_2 n$).

This yields the final time complexity for the Fast Fourier Transform:

$$T(n) = \mathcal{O}(n \log n)$$

9 Pointwise Multiplication

Once we have used the FFT to convert our polynomials $A(x)$ and $B(x)$ from the Coefficient Representation into the Value Representation, the actual multiplication step is trivial.

Because both polynomials are evaluated at the exact same n roots of unity, we simply multiply their corresponding y-values together to get the evaluated points of our resulting polynomial $C(x)$.

$$C(\omega^k) = A(\omega^k) \cdot B(\omega^k)$$

Since there are n points, this takes linear $\mathcal{O}(n)$ time. The final challenge is transforming $C(x)$ back into the Coefficient Representation.

When applying the FFT to multiply two polynomials of degree d, we must be very careful about our choice of n (the number of evaluation points).

Let $A(x)$ and $B(x)$ both be polynomials of degree d. Their product, $C(x) = A(x)B(x)$, will have a degree of $2d$. According to our core theorem, to uniquely reconstruct $C(x)$ during the interpolation step, we need at least $2d + 1$ evaluated points.

However, for our divide-and-conquer FFT algorithm to work perfectly down to the base case, the size of our input arrays must be a power of 2.

Therefore, we must choose n such that:

1. $n \geq 2d + 1$
2. $n = 2^k$ for some integer k

Zero-Padding

If our original polynomials only have $d+1$ coefficients, how do we evaluate them at n points where $n > d+1$? We simply pad the coefficient arrays with zeros up to length n. This does not change the value of the polynomials, but it satisfies the input size requirements for the FFT.

Example: If $A(x) = 2 + 3x + x^2$ (degree $d = 2$), we need $n \geq 2(2) + 1 = 5$. The next power of 2 is $n = 8$.

We pad the coefficient representation of $A(x)$ to length 8:

$$A = [2, 3, 1, 0, 0, 0, 0, 0]$$

By zero-padding both $A(x)$ and $B(x)$ to size n, we guarantee that the pointwise multiplication will yield n points, giving the Inverse FFT exactly enough information to reconstruct the $2d + 1$ coefficients of $C(x)$.

10 Interpolation & The DFT Matrix

Let us look at the evaluation step from an alternative perspective using linear algebra. Evaluating a polynomial $P(x) = p_0 + p_1 x + \cdots + p_{n-1}x^{n-1}$ at n points can be written as a matrix-vector multiplication.

When those n points are the nth roots of unity ($x_k = \omega^k$ where $\omega = e^{\frac{2\pi i}{n}}$), the system looks like this:

$$\begin{bmatrix} P(\omega^0) \\ P(\omega^1) \\ P(\omega^2) \\ \vdots \\ P(\omega^{n-1}) \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \dots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \dots & \omega^{(n-1)(n-1)} \end{bmatrix}}_{\text{Discrete Fourier Transform (DFT) matrix}} \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ \vdots \\ p_{n-1} \end{bmatrix}$$

The large Vandermonde matrix in this equation is famously known as the **Discrete Fourier Transform (DFT) matrix**. The FFT algorithm is simply a highly optimized way to compute this exact matrix-vector product.

11 The Inverse Matrix & Interpolation

Interpolation is the exact opposite process. We have the evaluated points (the vector on the left), and we want to find the coefficients (the vector on the right). Algebraically, this involves multiplying both sides by the **inverse** of the DFT matrix:

$$\begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ \vdots \\ p_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \dots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \dots & \omega^{(n-1)(n-1)} \end{bmatrix}^{-1} \begin{bmatrix} P(\omega^0) \\ P(\omega^1) \\ P(\omega^2) \\ \vdots \\ P(\omega^{n-1}) \end{bmatrix}$$

Inverting a large matrix is usually a computationally heavy $\mathcal{O}(n^3)$ operation. However, because of the geometric properties of the roots of unity, a mathematical miracle happens. The inverse of the DFT matrix looks almost exactly like the original matrix!

$$\begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ \vdots \\ p_{n-1} \end{bmatrix} = \frac{1}{n} \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^{-1} & \omega^{-2} & \dots & \omega^{-(n-1)} \\ 1 & \omega^{-2} & \omega^{-4} & \dots & \omega^{-2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{-(n-1)} & \omega^{-2(n-1)} & \dots & \omega^{-(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} P(\omega^0) \\ P(\omega^1) \\ P(\omega^2) \\ \vdots \\ P(\omega^{n-1}) \end{bmatrix}$$

Notice the pattern: Every ω in the original DFT matrix is simply replaced by ω^{-1} (its complex conjugate), and the entire matrix is scaled by $\frac{1}{n}$.

12 The Inverse FFT (IFFT)

Because the inverse matrix shares the exact same structure as the original DFT matrix, we do not need to invent a new algorithm for interpolation.

To understand why, let's look at the interpolation matrix equation again. Computing the original coefficients $[p_0, p_1, \dots, p_{n-1}]$ using the inverse matrix is mathematically equivalent to **evaluating a brand new polynomial**.

Let this new polynomial be $Q(x)$, where its "coefficients" are the evaluated values of our original polynomial $P(x)$:

$$Q(x) = P(\omega^0) + P(\omega^1)x + P(\omega^2)x^2 + \cdots + P(\omega^{n-1})x^{n-1}$$

If you look closely at the rows of the inverse DFT matrix, finding any original coefficient p_k is exactly the same as evaluating $Q(x)$ at the point ω^{-k} and scaling the result by $\frac{1}{n}$:

$$p_k = \frac{1}{n} Q(\omega^{-k})$$

13 The IFFT Algorithm

Algorithm 2 Inverse Fast Fourier Transform (IFFT)

function IFFT(P_{vals})
    ▷ $P_{vals} = [P(\omega^0), P(\omega^1), \ldots, P(\omega^{n-1})]$
    $n \leftarrow \text{len}(P_{vals})$
    $y \leftarrow$ IFFT_Helper(P_{vals}, n)
    ▷ Scale the final results by $1/n$
    for $j \leftarrow 0$ **to** $n-1$ **do**
        $y[j] \leftarrow y[j]/n$
    end for
    return y ▷ These are the original coefficients $[p_0, p_1, \ldots, p_{n-1}]$
end function

function IFFT_Helper(A, m)
    if $m = 1$ **then**
        return A
    end if
    $\omega \leftarrow e^{-2\pi i/m}$ ▷ Use the inverse root of unity (negative exponent)
    $A_e \leftarrow A[0 :: 2]$ ▷ Extract even-indexed elements
    $A_o \leftarrow A[1 :: 2]$ ▷ Extract odd-indexed elements
    $y_e \leftarrow$ IFFT_Helper($A_e, m/2$)
    $y_o \leftarrow$ IFFT_Helper($A_o, m/2$)
    $y \leftarrow [0] \times m$ ▷ Initialize output array of size m
    for $j \leftarrow 0$ **to** $m/2 - 1$ **do**
        $y[j] \leftarrow y_e[j] + \omega^j y_o[j]$
        $y[j + m/2] \leftarrow y_e[j] - \omega^j y_o[j]$
    end for
    return y
end function

Because interpolation is just another polynomial evaluation at the nth roots of unity (specifically using ω^{-1} instead of ω), we can feed it directly into our existing $\mathcal{O}(n \log n)$ recursive FFT algorithm. We just need two tiny modifications:

Evaluation (FFT):

- Input: Coefficients $[p_0, p_1, \ldots, p_{n-1}]$
- Base root of unity: $\omega = e^{\frac{2\pi i}{n}}$

Interpolation (Inverse FFT):

- Input (treated as coefficients of Q): $[P(\omega^0), P(\omega^1), \ldots, P(\omega^{n-1})]$
- Base root of unity: $\omega^{-1} = e^{-\frac{2\pi i}{n}}$
- Final step: **Multiply every resulting coefficient by** $\frac{1}{n}$

This profound symmetry allows us to complete the entire polynomial multiplication pipeline Evaluation, Pointwise Multiplication, and Interpolation using the same underlying algorithm in $\mathcal{O}(n \log n)$ time.